Rhilo Sotto
3/12/2025
COMPE 470L

Lab 5 Report

Introduction

In this lab we designed a UART Receiver to receive our last name at 9600 bit/s via keystrokes through PuTTY, which sends the 8-bit data package to the UART Transmitter we designed in the previous lab. The transmitter sends back the original keypress as a datastream, which shows up in the PuTTY terminal. We implemented it in Verilog alongside a top level module that included both receiver and transmitter to program into our FPGA. The transmitted stream has 8 data bits, alongside a start and a stop bit to indicate when transmission begins and ends.

Verilog Code

UART Transmitter Module

```
module uart_tx
  #(parameter CLKS_PER_BIT = 10)
  (
   input       i_Clock,
   input       i_Tx_DV,
   input [7:0] i_Tx_Byte,
   output      o_Tx_Active,
   output reg  o_Tx_Serial,
   output      o_Tx_Done
   );

  localparam s_IDLE         = 3'b000;
  localparam s_TX_START_BIT = 3'b001;
  localparam s_TX_DATA_BITS = 3'b010;
  localparam s_TX_STOP_BIT  = 3'b011;
  localparam s_CLEANUP      = 3'b100;

  reg [2:0]    r_SM_Main     = 0;
  reg [14:0]    r_Clock_Count = 0;
  reg [2:0]    r_Bit_Index   = 0;
  reg [7:0]    r_Tx_Data     = 0;
  reg          r_Tx_Done     = 0;
  reg          r_Tx_Active   = 0;
```

```
always @(posedge i_Clock)
  begin

    case (r_SM_Main)
      s_IDLE :
        begin
          o_Tx_Serial   <= 1'b1;         // Drive Line High for Idle
          r_Tx_Done     <= 1'b0;
          r_Clock_Count <= 0;
          r_Bit_Index   <= 0;

          if (i_Tx_DV == 1'b1)
            begin
              r_Tx_Active <= 1'b1;
              r_Tx_Data   <= i_Tx_Byte;
              r_SM_Main   <= s_TX_START_BIT;
            end
          else
            r_SM_Main <= s_IDLE;
        end // case: s_IDLE


      // Send out Start Bit. Start bit = 0
      s_TX_START_BIT :
        begin
          o_Tx_Serial <= 1'b0;

          // Wait CLKS_PER_BIT-1 clock cycles for start bit to finish
          if (r_Clock_Count < CLKS_PER_BIT-1)
            begin
              r_Clock_Count <= r_Clock_Count + 1;
              r_SM_Main     <= s_TX_START_BIT;
            end
          else
            begin
              r_Clock_Count <= 0;
              r_SM_Main     <= s_TX_DATA_BITS;
            end
        end // case: s_TX_START_BIT


      // Wait CLKS_PER_BIT-1 clock cycles for data bits to finish
      s_TX_DATA_BITS :
```

```
  begin
    o_Tx_Serial <= r_Tx_Data[r_Bit_Index];

    if (r_Clock_Count < CLKS_PER_BIT-1)
      begin
        r_Clock_Count <= r_Clock_Count + 1;
        r_SM_Main     <= s_TX_DATA_BITS;
      end
    else
      begin
        r_Clock_Count <= 0;

        // Check if we have sent out all bits
        if (r_Bit_Index < 7)
          begin
            r_Bit_Index <= r_Bit_Index + 1;
            r_SM_Main   <= s_TX_DATA_BITS;
          end
        else
          begin
            r_Bit_Index <= 0;
            r_SM_Main   <= s_TX_STOP_BIT;
          end
      end
  end // case: s_TX_DATA_BITS


// Send out Stop bit.  Stop bit = 1
s_TX_STOP_BIT :
  begin
    o_Tx_Serial <= 1'b1;

    // Wait CLKS_PER_BIT-1 clock cycles for Stop bit to finish
    if (r_Clock_Count < CLKS_PER_BIT-1)
      begin
        r_Clock_Count <= r_Clock_Count + 1;
        r_SM_Main     <= s_TX_STOP_BIT;
      end
    else
      begin
        r_Tx_Done     <= 1'b1;
        r_Clock_Count <= 0;
        r_SM_Main     <= s_CLEANUP;
        r_Tx_Active   <= 1'b0;
```

```
            end
          end // case: s_Tx_STOP_BIT


        // Stay here 1 clock
        s_CLEANUP :
          begin
            r_Tx_Done <= 1'b1;
            r_SM_Main <= s_IDLE;
          end


        default :
          r_SM_Main <= s_IDLE;

      endcase
    end

  assign o_Tx_Active = r_Tx_Active;
  assign o_Tx_Done   = r_Tx_Done;

endmodule
```

UART Receiver Module

```
module uart_rx
  #(parameter CLKS_PER_BIT = 10)
  (
   input        i_Clock,
   input        i_Rx_Serial,
   output       o_Rx_DV,
   output reg [7:0] o_Rx_Byte
   );

  localparam s_IDLE         = 3'b000;
  localparam s_RX_START_BIT = 3'b001;
  localparam s_RX_DATA_BITS = 3'b010;
  localparam s_RX_STOP_BIT  = 3'b011;
  localparam s_CLEANUP      = 3'b100;

  reg          r_Rx_Data_R = 1'b1;
  reg          r_Rx_Data   = 1'b1;

  reg [14:0]     r_Clock_Count = 0;
  reg [2:0]     r_Bit_Index   = 0; //8 bits total
  reg [7:0]     r_Rx_Byte     = 0;
  reg          r_Rx_DV       = 0;
  reg [2:0]     r_SM_Main     = 0;

  // Double-register the incoming data.

//  always @(posedge i_Clock)
//    begin
//      r_Rx_Data_R <= i_Rx_Serial;
//      r_Rx_Data   <= r_Rx_Data_R;
//    end


  // Control RX state machine
  always @(posedge i_Clock)
    begin

      case (r_SM_Main)
        s_IDLE :
          begin
            r_Rx_DV       <= 1'b0;
            r_Clock_Count <= 0;
```

```
    r_Bit_Index   <= 0;

    if (i_Rx_Serial == 1'b0)          // Start bit detected
      r_SM_Main <= s_RX_START_BIT;
    else
      r_SM_Main <= s_IDLE;
  end

// Check middle of start bit to make sure it's still low
s_RX_START_BIT :
  begin
    if (r_Clock_Count == (CLKS_PER_BIT-1)/2)
      begin
        if (i_Rx_Serial == 1'b0)
          begin
            r_Clock_Count <= 0;  // reset counter, found the middle
            r_SM_Main     <= s_RX_DATA_BITS;
          end
        else
          r_SM_Main <= s_IDLE;
      end
    else
      begin
        r_Clock_Count <= r_Clock_Count + 1;
        r_SM_Main     <= s_RX_START_BIT;
      end
  end // case: s_RX_START_BIT


// Wait CLKS_PER_BIT-1 clock cycles to sample serial data
s_RX_DATA_BITS :
  begin
    if (r_Clock_Count < CLKS_PER_BIT-1)
      begin
        r_Clock_Count <= r_Clock_Count + 1;
        r_SM_Main     <= s_RX_DATA_BITS;
      end
    else
      begin
        r_Clock_Count          <= 0;
        o_Rx_Byte[r_Bit_Index] <= i_Rx_Serial;

        // Check if we have received all bits
        if (r_Bit_Index < 7)
```

```
            begin
              r_Bit_Index <= r_Bit_Index + 1;
              r_SM_Main   <= s_RX_DATA_BITS;
            end
          else
            begin
              r_Bit_Index <= 0;
              r_SM_Main   <= s_RX_STOP_BIT;
            end
        end
    end // case: s_RX_DATA_BITS


  // Receive Stop bit.  Stop bit = 1
  s_RX_STOP_BIT :
    begin
      // Wait CLKS_PER_BIT-1 clock cycles for Stop bit to finish
      if (r_Clock_Count < CLKS_PER_BIT-1)
        begin
          r_Clock_Count <= r_Clock_Count + 1;
          r_SM_Main     <= s_RX_STOP_BIT;
        end
      else
        begin
          r_Rx_DV       <= 1'b1;
          r_Clock_Count <= 0;
          r_SM_Main     <= s_CLEANUP;
        end
    end // case: s_RX_STOP_BIT


  // Stay here 1 clock
  s_CLEANUP :
    begin
      r_SM_Main <= s_IDLE;
      r_Rx_DV   <= 1'b0;
    end


  default :
    r_SM_Main <= s_IDLE;

 endcase
end
```

```
  assign o_Rx_DV   = r_Rx_DV;
  //assign o_Rx_Byte = r_Rx_Byte;

endmodule // uart_rx
```

UART Top Level Module

```
module uart_tb (
    input clk_100MHz,       // basys 3 FPGA clock signal
    input rx,               // USB-RS232 Rx
    output tx               // USB-RS232 Tx
);

    // 100MHz Clock / 9600 Baudrate = 10417
  localparam c_CLKS_PER_BIT    = 10417;

  wire w_Tx_Done, tx_ACTIVE;
  wire rx_DV;
  wire [7:0] w_Rx_Byte;

  uart_rx #(.CLKS_PER_BIT(c_CLKS_PER_BIT)) UART_RX_INST
    (.i_Clock(clk_100MHz),
     .i_Rx_Serial(rx),
     .o_Rx_DV(rx_DV),
     .o_Rx_Byte(w_Rx_Byte)
     );

  uart_tx #(.CLKS_PER_BIT(c_CLKS_PER_BIT)) UART_TX_INST
    (.i_Clock(clk_100MHz),
     .i_Tx_DV(rx_DV),
     .i_Tx_Byte(w_Rx_Byte),
     .o_Tx_Active(tx_ACTIVE),
     .o_Tx_Serial(tx),
     .o_Tx_Done(w_Tx_Done)
     );

endmodule
```

Constraint File

```
# Clock signal
set_property PACKAGE_PIN W5 [get_ports clk_100MHz]

        set_property IOSTANDARD LVCMOS33 [get_ports clk_100MHz]
        create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports
clk_100MHz]

##USB-RS232 Interface
set_property PACKAGE_PIN B18 [get_ports rx]
set_property IOSTANDARD LVCMOS33 [get_ports rx]
set_property PACKAGE_PIN A18 [get_ports tx]
set_property IOSTANDARD LVCMOS33 [get_ports tx]
```

Results

Conclusion

In this lab, we learned how to design a UART Receiver to receive our last name at 9600 bit/s in Verilog and successfully calculated the required clock cycles per second for a baud rate of 9600 bit/s, given a 100MHz clock frequency such as that on the Basys3. I successfully showcased my last name in uppercase ASCII in the PuTTY terminal, and learned how to use PuTTY to interface with the Basys3 RX and TX pins.